

## Opción 1 — *PaaS*

### AWS → Elastic Beanstalk (Tomcat)

- Subes tu `.war` directamente.
- AWS crea:
  - EC2
  - Load Balancer
  - Auto Scaling
  - Tomcat configurado
- Ideal si no quieres gestionar servidores.

#### Cuándo usarla

- ✓ Quieres algo rápido
  - ✓ No necesitas mucha personalización del SO
  - ✓ Carga moderada o variable
- 

### Google Cloud → App Engine Standard (Java)

- Subes el `.war` o el proyecto Java.
- Google gestiona todo.
- Autoescalado automático.

#### Cuándo usarla

- ✓ Quieres cero administración
  - ✓ Coste bajo al inicio
  - ✓ Tráfico irregular
- 

## Opción 2 — *Servidor en la nube*

### AWS → EC2 + Tomcat

- Creas una VM EC2.
- Instalas Java + Tomcat.
- Copias el `.war` a `/webapps`.

#### Pros

- Control total.

- Fácil de depurar.
- Barato para proyectos pequeños.

#### **Contras**

- Gestionas parches, reinicios, backups.
- 

#### **Google Cloud → Compute Engine + Tomcat**

- Igual que EC2 pero en GCP.
- 

### **Opción 3 — *Docker***

#### **AWS → ECS / EKS**

#### **Google Cloud → Cloud Run / GKE**

- Metes el `.war` dentro de una imagen Docker con Tomcat o Jetty.
- Lo despliegas como contenedor.

#### **Pros**

- Mismo artefacto en AWS y GCP.
- Escalado limpio.
- Ideal si piensas crecer.

#### **Contras**

- Más complejidad inicial.